

Programación Orientado a Objetos (Herencia y Relaciones)

Natalia Gallego Barrero
Juan Carlos Mejía Cano
Docente: Alejandro Rodas Vásquez
Universidad Tecnológica de Pereira

5 de mayo de 2026

1. Introducción

El presente proyecto tiene como objetivo el desarrollo de un sistema de facturación para una tienda agrícola, el cual permite gestionar diferentes tipos de productos como fertilizantes, plaguicidas y antibióticos para animales.

Este sistema surge de la necesidad de organizar la información relacionada con la venta de productos agrícolas, permitiendo llevar un control sobre los productos disponibles, los clientes y las facturas generadas.

El sistema permite:

- Registrar productos
- Realizar compras
- Generar facturas

Además, se aplican conceptos de Programación Orientada a Objetos como herencia, encapsulamiento y composición.

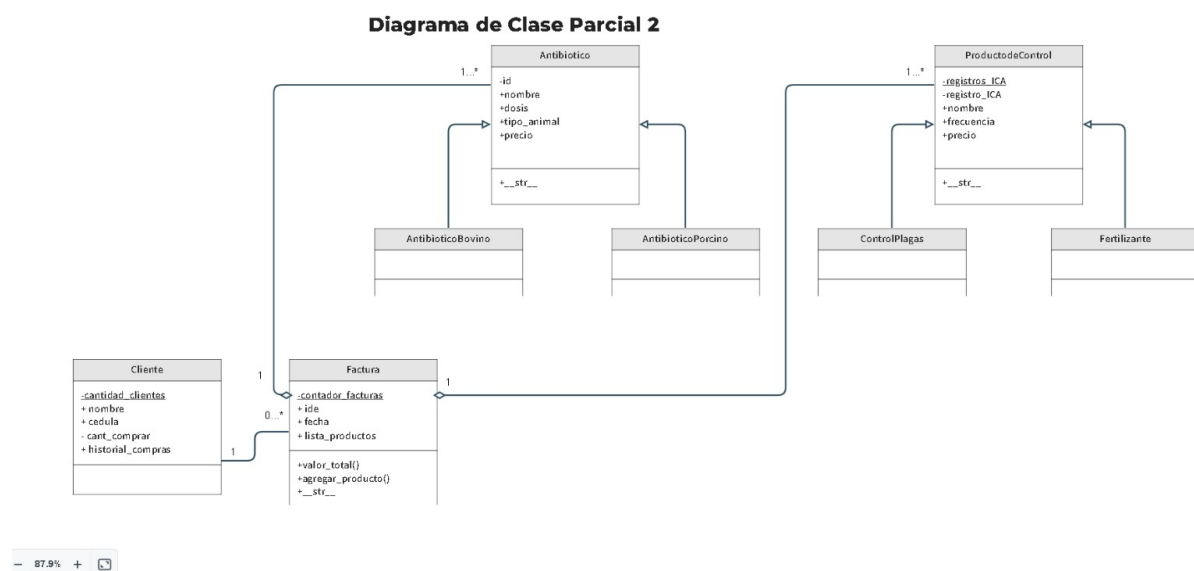
2. Diseño del Sistema

2.1. Clases principales

- **Cliente:** Representa a una persona que realiza compras en la tienda.
- **Factura:** Representa el comprobante de compra, el cual contiene los productos adquiridos.
- **ProductoControl:** Clase padre para los productos agrícolas.
- **ControlPlagas:** Clase hija de ProductoControl, especializada en plaguicidas.
- **Fertilizante:** Clase hija de ProductoControl, especializada en fertilizantes.
- **Antibiotico:** Clase padre para medicamentos veterinarios.
- **AntibioticoBovino:** Clase hija de Antibiotico.
- **AntibioticoPorcino:** Clase hija de Antibiotico.

Cada clase fue diseñada para representar entidades reales dentro de una tienda agrícola, manteniendo coherencia con los requerimientos del problema.

2.2. Diagrama UML



Descripción: Se evidencia la relación de herencia entre las clases padre e hijas, así como la composición entre Factura y los productos.

3. Herencia

La herencia es un mecanismo de la programación orientada a objetos que permite que una clase (hija) herede atributos y métodos de otra clase (padre), evitando la duplicación de código.

3.1. Antibióticos

En este caso, se implementa una jerarquía donde:

- **Clase padre:** Antibiotico
- **Clases hijas:** AntibioticoBovino, AntibioticoPorcino

La clase padre contiene atributos comunes como:

- nombre
- dosis
- precio

Las clases hijas especializan el atributo **tipo__animal**, permitiendo diferenciar entre bovinos y porcinos.

3.2. Productos de Control

De manera similar, se implementa otra jerarquía:

- **Clase padre:** ProductoControl
- **Clases hijas:** ControlPlagas, Fertilizante

La clase padre contiene atributos como:

- registro ICA
- nombre
- frecuencia
- valor

Mientras que las clases hijas agregan atributos específicos:

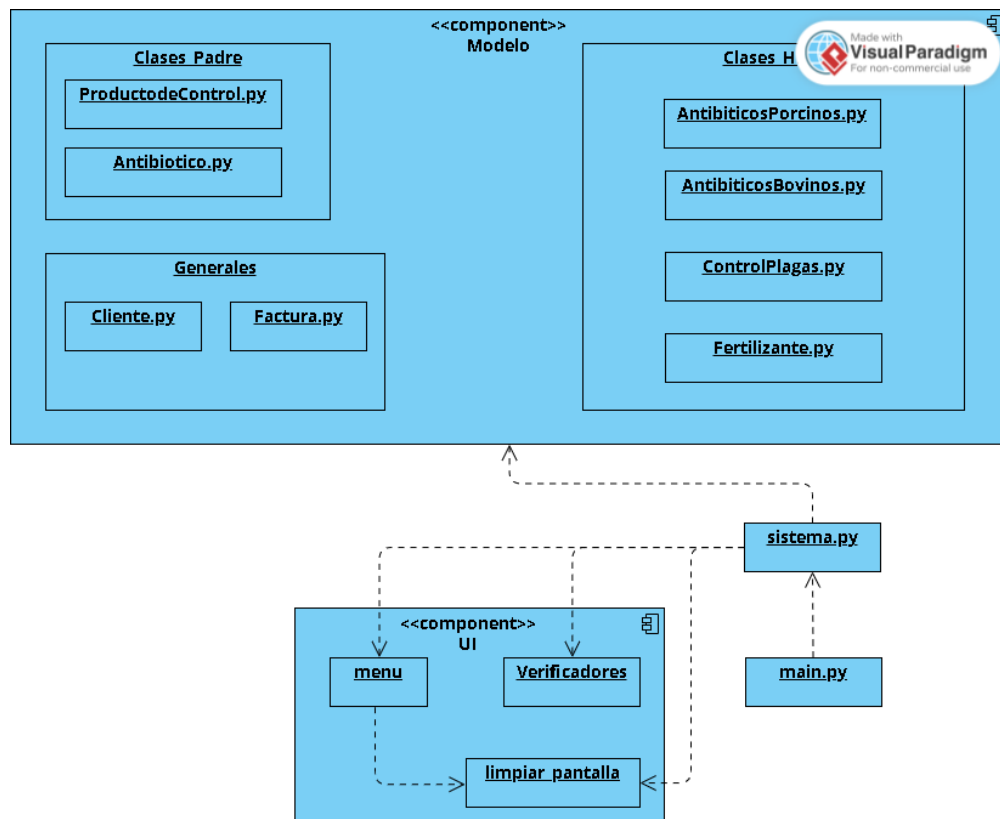
- ControlPlagas: periodo de carencia
- Fertilizante: fecha de última aplicación

4. Composición

La composición es una relación en la cual una clase contiene objetos de otra clase.

En este proyecto, la clase Factura tiene una relación de composición con los productos, ya que una factura puede contener múltiples productos.

Evidencia:



Explicación:

En la imagen se observa que la factura contiene una lista de productos (`lista_productos`), lo que demuestra que un objeto Factura está compuesto por varias instancias de productos.

Esto permite modelar correctamente una compra real, donde un cliente adquiere múltiples productos en una sola transacción.

5. Pruebas Unitarias

Se realizaron pruebas con el objetivo de verificar el correcto funcionamiento del sistema.

Las pruebas incluyen:

- Creación de objetos de diferentes tipos.
- Agregado de productos a la factura.
- Cálculo correcto del valor total.

Evidencia:

```
12 # ===== TEST 1 =====
13
14
15 def test_agregar_productos_factura():
16     factura = Factura()
17
18     a = AntibioticoBovino("Oxitetraciclina", 10, 20000)
19     p = ControlPlagas(1234, "Glifosato", 15, 50000, 7)
20
21     factura.agregar_producto(a)
22     factura.agregar_producto(p)
23
24     assert factura.valor_total == 70000
25
26     print("\nTest 1: Factura suma productos correctamente\n")
27
28
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main> & C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main\pruebas.py
Test 1: Factura suma productos correctamente
```

Test 1

```
27 # ===== TEST 2 =====
28
29 def test_herencia_antibioticos():
30     bovino = AntibioticoBovino("Penicilina", 5, 30000)
31     porcino = AntibioticoPorcino("Tilosina", 8, 25000)
32
33     assert bovino.tipo_animal == "bovino"
34     assert porcino.tipo_animal == "porcino"
35
36     print("\nTest 2: Herencia de antibióticos correcta\n")
37
38
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main> & C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main\pruebas.py
Test 2: Herencia de antibióticos correcta
```

Test 2

```
40 # ===== TEST 3 =====
41
42 def test_producto_control():
43     plaga = ControlPlagas(5678, "Mancozeb", 10, 40000, 14)
44
45     assert plaga._registroICA == 5678
46     assert plaga.periodo_carencia == 14
47
48     print("\nTest 3: Producto de control correcto\n")
49
50
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main> & C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main\pruebas.py
Test 3: Producto de control correcto
```

Test 3

```
49 # ===== TEST 4 =====
50
51 def test_fertilizante():
52     fert = Fertilizante(9999, "Urea", 30, 80000, "10/05/2026")
53
54     assert fert.nombre == "Urea"
55     assert fert.frecuencia == 30
56
57     print("\nTest 4: Fertilizante correcto\n")
58
59
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main> & C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main\pruebas.py
Test 4: Fertilizante correcto
```

Test 4

```
62
63 # ===== TEST 5 =====
64
65 def test_factura_vacia():
66     factura = Factura()
67     assert factura.valor_total == 0
68     assert len(factura.lista_productos) == 0
69
70     print("\nTest 5: Factura vacía correcta\n")
71
72
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main> & C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main/pruebas.py

Test 5: Factura vacía correcta

Test 5

```
72
73 # ===== TEST 6 =====
74
75 def test_multiples_productos():
76     factura = Factura()
77
78     a = AntibioticoBovino("A", 1, 10000)
79     b = AntibioticoPorcino("B", 1, 20000)
80     c = ControlPlagas(1111, "C", 10, 30000, 5)
81
82     factura.agregar_producto(a)
83     factura.agregar_producto(b)
84     factura.agregar_producto(c)
85
86     assert factura.valor_total == 60000
87
88     print("\nTest 6: Múltiples productos correcto\n")
89
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main> & C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main/pruebas.py

Test 6: Múltiples productos correcto

Test 6

```
90
91 # ===== TEST 7 =====
92
93 from sistema import Sistema
94
95 def test_búsqueda_producto():
96     sistema = Sistema()
97
98     p = ControlPlagas(1234, "Test", 10, 10000, 5)
99
100     Sistema._Sistema__productos += 1
101     sistema.productos['productos_control']['plagas'][Sistema._Sistema__productos] = p
102
103     encontrado = sistema.buscar_producto_por_id(Sistema._Sistema__productos)
104
105     assert encontrado is not None
106
107     print("\nTest 7: Búsqueda correcta\n")
108
109
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main> & C:\Users\natyg\AppData\Local\Programs\Python\Python39\python.exe C:\Users\natyg\Downloads\Parcial2_Parte1_Herencia_Relaciones-main/pruebas.py

Test 7: Búsqueda correcta

Test 7

Explicación:

Los resultados muestran que el sistema funciona correctamente, ya que no se presentan errores durante la ejecución de las pruebas y los cálculos son consistentes.

6. Debug

Se utilizó el depurador para analizar el comportamiento del sistema en tiempo de ejecución.

Evidencia:



```
▼ productos = {'productos_control': {'plagas': {...}, 'fertilizantes': {}}, 'anti... 1
  > special variables 1
  > function variables 1
  ▼ 'productos_control' = {'plagas': {1: <Modelo.ClasesHijas.ControlPlagas.Control... 1
    > special variables 1
    > function variables 1
    > 'plagas' = {1: <Modelo.ClasesHijas.ControlPlagas.ControlPlagas object at 0x00... 1
    > 'fertilizantes' = {} 1
    len() = 2
  ▼ 'antibioticos' = {'porcinos': {2: <Modelo.ClasesHijas.AntibioticosPorcinos.Anti... P
    > special variables
    > function variables
    ▼ 'porcinos' = {2: <Modelo.ClasesHijas.AntibioticosPorcinos.AntibioticoPorcino ... =
      > special variables
      > function variables
      ▼ 2 = <Modelo.ClasesHijas.AntibioticosPorcinos.AntibioticoPorcino object at 0x...
        > special variables
        dosis = 1
        nombre = 'Ceftiofur Sódico 4 g'
        precio = 115000
        tipo_animal = 'porcino'
        _Antibiotico_registros_antibiotico = 1
        _id = 1
        len() = 1
      > 'bovinos' = {}
      len() = 2
      len() = 2
      _Sistema_productos = 2
```

Explicación:

En la imagen del debug se observa un objeto de tipo AntibioticoPorcino. Este objeto contiene atributos como nombre, dosis y precio, los cuales no fueron definidos directamente en la clase hija, sino que son heredados de la clase padre Antibiotico.

Esto demuestra que la herencia permite reutilizar la estructura base de una clase y extenderla con características específicas.

Evidencia:

```
1
  lista_productos = [<Modelo.ClasesHijas.ControlPlagas.ControlPlagas object at 0...
> special variables
> function variables
  0 = <Modelo.ClasesHijas.ControlPlagas.ControlPlagas object at 0x00000278CFDC0...
  > special variables
    frecuencia = 10
    nombre = 'Mancozeb (80% polvo mojable)'
    periodo_carencia = 14
    valor = 35000
    _registroICA = 1378
  1 = <Modelo.ClasesHijas.AntibioticosPorcinos.AntibioticoPorcino object at 0x0...
  > special variables
    dosis = 1
    nombre = 'Ceftiofur Sódico 4 g'
    precio = 115000
    tipo_animal = 'porcino'
    _Antibiotico_registros_antibiotico = 1
    _id = 1
    len() = 2
    valor_total = 150000
    _contador_facturas = 1
```

Explicación: En el debug se puede observar la estructura interna de los objetos, incluyendo sus atributos y relaciones.

7. Conclusiones

A partir del desarrollo del sistema de facturación para la tienda agrícola, se pueden establecer las siguientes conclusiones:

- La implementación de la **herencia** permitió estructurar el sistema de manera jerárquica, facilitando la reutilización de código y evitando la duplicación de atributos en las clases hijas. Esto se evidenció en las clases de antibióticos y productos de control, donde se definieron atributos comunes en las clases padre y se especializaron en las clases hijas.
- La correcta definición de **atributos y métodos** en cada clase permitió representar de manera adecuada los elementos del mundo real, como clientes, productos y transacciones, logrando un sistema coherente y funcional.
- Finalmente, este proyecto permitió afianzar los conceptos de la programación orientada a objetos, demostrando su importancia en el desarrollo de sistemas estructurados, mantenibles y escalables.